

Sign-Language Landmark Pipeline

Technical Documentation & System Design

Generated 2026-06-24

#	Section
01	Overview
02	Architecture
03	Pipeline
04	File Reference
05	Data Formats
06	Developer Guide
07	Future Design
A	Appendix — Verification Infographics

01 — Overview

What this system does

It turns a folder of sign-language **video clips** into a **machine-learning-ready landmark dataset**, and then **proves** that the extracted data is correct.

For every clip it runs Google MediaPipe to detect three things per frame — the **face mesh** (478 points), **both hands** (21 points each) and the **upper-body pose** (33 points) — smooths them, and saves them as NumPy arrays. It also renders a black-background skeleton video and an overlay video so the numeric data can be checked by eye, and it produces QA reports and infographics.

The original detector (`landmark_detector.py`) only **displayed** landmarks live; this project adds the **dataset extraction, verification and reporting** layer on top of it.

Why each piece exists

- **Extraction** (`extract_dataset.py`) — the data you actually train on.
- **Verification** (`verify_landmarks.py`) — answers "are the `.npy` files really correct?" with structural checks, a consistency proof, and an eyes-on overlay.
- **Pruning** (`prune_clips.py`) — removes bad/outlier clips reversibly.
- **Reporting** (`dataset_report.py`) — infographics that summarise quality.

The dataset

- **12 word classes**: ``good, hello, help, hi, how_are_you, me, nice_to_meet_you, no, sorry, thank_you, yes, you``.
- **134 clips** after pruning (136 filmed; 2 close-up outliers excluded).
- Source clips are 1920x1080 @ 30 fps, ~14–286 frames each.

Tech stack

Layer	Tool	Version (tested)
Language	Python	3.11
Detection	MediaPipe	0.10.9
Video / image I/O	OpenCV (<code>opencv-python</code>)	4.13
Arrays	NumPy	2.4
Charts	matplotlib	3.10
PDF docs	reportlab	5.0

Directory tree

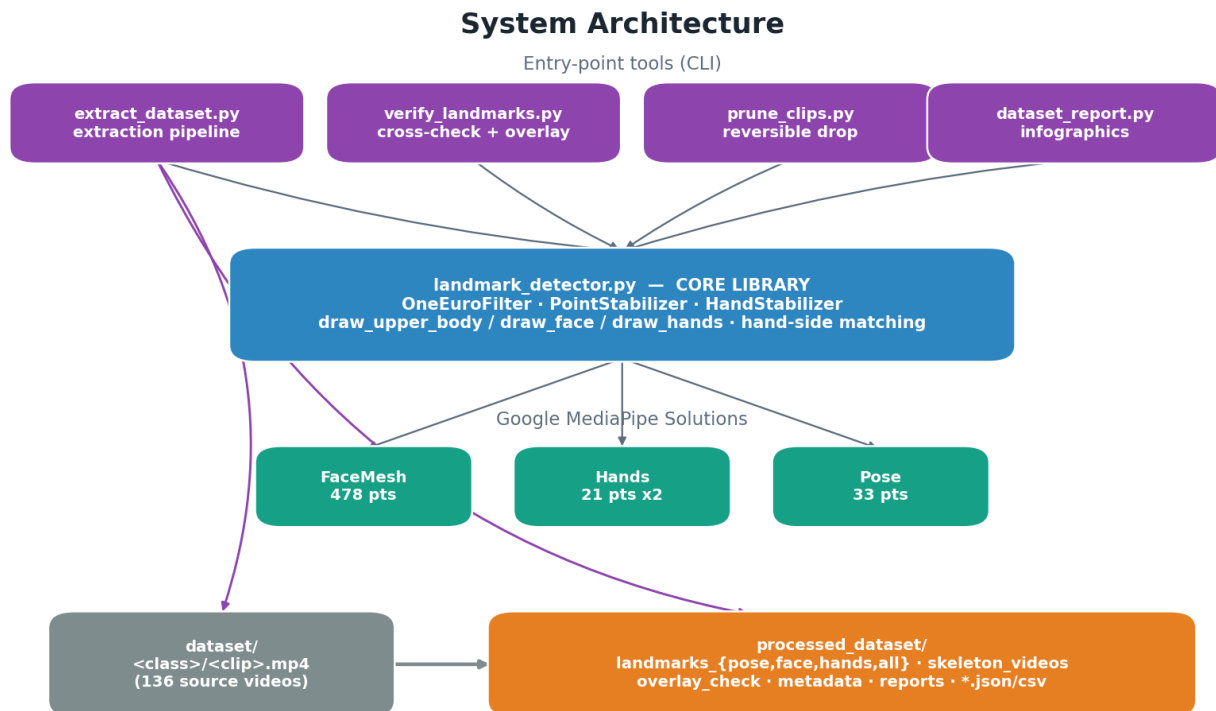
```
sign new exprmt/
+-- landmark_detector.py      # CORE library: detection, smoothing, drawing (also a live viewer)
+-- extract_dataset.py        # Pipeline: dataset/ -> processed_dataset/
+-- verify_landmarks.py       # Cross-checks the .npy files (+ overlay videos)
+-- prune_clips.py            # Reversibly drop clips from the dataset
+-- dataset_report.py         # Verification infographics
|
+-- dataset/                  # SOURCE videos (input)
|   +-- good/ *.mp4
|   +-- hello/ *.mp4
```

```

| +-- ... (12 classes)
|
+-- excluded_clips/                # Clips pruned out (preserved, not deleted)
| +-- no/no_13.mp4
| +-- yes/yes_12.mp4
|
+-- processed_dataset/            # ALL OUTPUTS
| +-- landmarks_pose/<class>/<stem>.npz    # (T, 33, 4)
| +-- landmarks_face/<class>/<stem>.npz    # (T, 478, 3)
| +-- landmarks_hands/<class>/<stem>.npz   # (T, 2, 21, 3)
| +-- landmarks_all/<class>/<stem>.npz     # (T, 1692) combined
| +-- skeleton_videos/<class>/<stem>.mp4   # black-bg skeleton
| +-- overlay_check/<class>/<stem>.mp4    # original + landmarks, side by side
| +-- metadata/<class>/<stem>.json        # per-clip QA
| +-- reports/*.png                    # infographics
| +-- FEATURE_LAYOUT.json              # layout of the (T,1692) vector
| +-- classes.json                    # class -> integer label
| +-- manifest.csv                    # one row per clip
| +-- extraction_report.json           # dataset-wide extraction QA
| +-- verification_report.json         # dataset-wide verification result
| +-- README.md
|
+-- docs/                            # THIS DOCUMENTATION
| +-- *.md
| +-- diagrams/*.png
| +-- TECHNICAL_DOCUMENTATION.pdf

```

02 — Architecture



Layers

The system is deliberately layered so the detection logic is written once and reused everywhere.

1. **MediaPipe Solutions** (third-party) — `FaceMesh`, `Hands`, `Pose`. These do the actual neural-network landmark detection on each RGB frame.
2. **Core library** — `landmark_detector.py` — wraps MediaPipe and adds the project's own logic: temporal smoothing (One-Euro filter), occlusion persistence, anatomical hand↔body matching, and all the drawing routines. It is also a standalone live viewer (`python landmark_detector.py`).
3. **Entry-point tools** — small CLIs that each do one job and all build on the core library:
 - `extract_dataset.py` — batch-extract the dataset.
 - `verify_landmarks.py` — verify and visualise the output.
 - `prune_clips.py` — curate the dataset.
 - `dataset_report.py` — summarise quality as charts.
4. **Data stores** — `dataset/` (input videos) and `processed_dataset/` (every output artifact).

Module responsibilities

Module	Responsibility	Depends on
<code>landmark_detector.py</code>	Detect, smooth, match, draw. Reusable building blocks.	MediaPipe, OpenCV
<code>extract_dataset.py</code>	Walk dataset, run detection, save <code>.npy</code> + skeleton + QA.	core lib, NumPy
<code>verify_landmarks.py</code>	Structure / consistency / detection checks + overlay.	core lib, NumPy
<code>prune_clips.py</code>	Move source out, delete artifacts, refresh reports.	stdlib only
<code>dataset_report.py</code>	Read metadata/reports, render infographics.	matplotlib, NumPy

Design principles

- **Single detection pass per clip.** Face, hands and pose are extracted

together, then sliced into the separate folders `*and*` concatenated into the combined vector. The per-modality files and the combined file therefore can never disagree (verified: max diff `0.0`).

- **Verification is rendered from the saved data, not re-detected.** The

skeleton and overlay videos are drawn *from the `.npy` arrays*, so watching them is literally a view of the file contents — a true proof, not a second opinion.

- **Honest masks.** Occlusion "hold" is disabled during extraction, so a frame

with no detection is stored as zeros and flagged in the mask, rather than back-filled with stale points. Detection rates are therefore truthful.

- **Reversibility.** Pruning *moves* source videos to `excluded_clips/` instead

of deleting them; the operation can be undone.

- **Reproducibility.** Every output folder is regenerable from `dataset/` by

re-running the tools. Nothing in `processed_dataset/` is hand-edited.

Coordinate & feature conventions

All landmarks are MediaPipe **normalised image coordinates**: `x, y` in `[0, 1]` (fraction of width/height), `z` a relative depth in roughly the same scale, and pose landmarks additionally carry a **visibility** in `[0, 1]`. These raw normalised coordinates *are* the spatial features; any further normalisation (centering, scaling) is left to the training pipeline and applied read-only from the files.

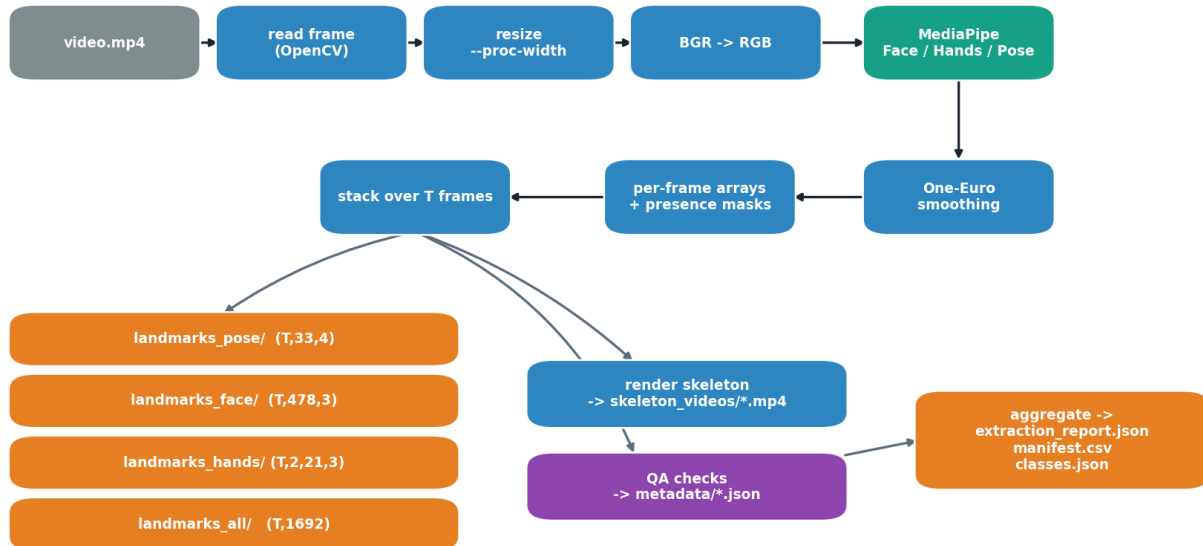
Note: pose `x/y` can legitimately fall outside `[0, 1]` — MediaPipe

extrapolates joints that are off-screen (e.g. hips below an upper-body frame).

Range checks therefore apply to face/hands only.

03 — Processing Pipeline

Extraction Data Flow (per clip)



One MediaPipe pass per clip feeds every output — the separate folders and the combined vector never diverge.

This is what happens for **one clip** inside `extract_dataset.py` (`extract_clip()`), and how clips are aggregated.

Per-clip steps

1. **Open the video** with OpenCV; read the source FPS (clamped to a sane range, default 30).
2. **Create fresh MediaPipe models** for this clip (`build_models`). Models are recreated per clip so temporal tracking state never leaks from one clip into the first frames of the next.
3. **Create fresh smoothing state** — one `PointStabilizer` for pose and a `HandStabilizer` for hands (`hold_frames=0`, so missing detections are not back-filled). Disabled by `--no-smooth`.
4. **For each frame** (timestamp `t = frame_index / fps`):
 - Optionally **downscale** to `--proc-width` (normalised coords are resolution-independent, so this only changes speed).
 - Convert **BGR -> RGB**.
 - Run **FaceMesh / Hands / Pose** (whichever are enabled).
 - **Pose** -> smooth with One-Euro -> write `(33, 4)` row `[x, y, z, visibility]`; set the pose mask for this frame if detected.
 - **Face** (first face only) -> write `(478, 3)` `[x, y, z]`; set the face mask.
 - **Hands** -> each detected hand is mapped to an **anatomical slot**

(Left = 0, Right = 1) via `anatomical_label()`; write `(21, 3)` into that slot and set the corresponding hand mask. Missing hand = zeros, mask 0.

5. **Stack** all per-frame arrays over `T` frames.

6. **Save the four .npy files:** `pose (T, 33, 4)`, `face (T, 478, 3)`, `hands (T, 2, 21, 3)`, and the **combined all** `(T, 1692)` built by `build_all_vector()` (concatenation order: pose | face | left | right).

7. **Render the skeleton video** (`render_skeleton`) from the *saved arrays* onto a black canvas, reusing the core library's `draw_*` functions.

8. **Run QA** (`qa_clip`): detection rates, coordinate ranges, NaN/Inf check, and a `PASS / WARN / FAIL` verdict. Write `metadata/<class>/<stem>.json`.

Aggregation

After all clips: write `classes.json` (class->label), `FEATURE_LAYOUT.json` (column layout of the combined vector), `manifest.csv` (one row per clip), `extraction_report.json` (dataset-wide verdicts + mean detection rates), and a `README.md`.

Key algorithms (in `landmark_detector.py`)

One-Euro filter (`OneEuroFilter`)

An adaptive low-pass filter. It smooths hard when a point is nearly still (kills jitter) and barely filters when a point moves fast (no lag). Each landmark's `x` and `y` are filtered independently. A large jump resets the filter so it does not smear across a teleport.

Point stabiliser (`PointStabilizer`)

Wraps one One-Euro pair per landmark for a fixed-size set (e.g. 33 pose points), plus optional **occlusion hold** (kept at 0 for extraction) and EMA-smoothed visibility. Returns a MediaPipe `NormalizedLandmarkList` proto.

Hand stabiliser (`HandStabilizer`)

Keeps one `PointStabilizer` per hand, keyed by MediaPipe's handedness label, so each hand is smoothed independently across frames.

Anatomical hand assignment (`anatomical_label, match_hands_to_sides`)

MediaPipe reports handedness assuming a mirrored (selfie) image. For raw video the label is swapped to the person's true side. `match_hands_to_sides()` then assigns each detected hand to the nearest body wrist using an optimal two-hand pairing, so the arm chains never cross into an "X".

Skeleton rendering from arrays (`render_skeleton`)

Rebuilds proto objects from the saved arrays and calls the same `draw_upper_body` / `draw_face` / `draw_hands` used by the live viewer (`show_labels=False` for clean output). Because it draws the saved numbers, the video is a faithful visualisation of the `.npy` file.

04 — File Reference

Every Python file, what it contains, and the logic of its key pieces.

landmark_detector.py — core library + live viewer (~637 lines)

The original, reused everywhere. Top-level constants define MediaPipe namespaces, pose landmark indices, the curated upper-body connections, tunables (`VIS_THRESH`, `HOLD_FRAMES`, One-Euro params) and drawing colours.

Symbol	Role
<code>OneEuroFilter</code>	Adaptive low-pass filter for a single scalar (<code>__call__(x, t)</code>).
<code>PointStabilizer</code>	Smooths a fixed-size landmark set; optional occlusion hold; tracks visibility. Returns a <code>NormalizedLandmarkList</code> .
<code>HandStabilizer</code>	One <code>PointStabilizer</code> per hand, keyed by handedness label.
<code>anatomical_label(raw, mirrored)</code>	Map MediaPipe handedness to the person's true side.
<code>match_hands_to_sides(wrists, lw, rw)</code>	Optimal hand->body-side assignment (no arm crossing).
<code>draw_upper_body(frame, proto, stale, wrists, show_labels=True)</code>	Torso + arm chains. <code>show_labels</code> added for clean dataset renders.
<code>draw_face / draw_hands</code>	Face mesh and hand skeletons via <code>mp_drawing</code> .
<code>run(...)</code>	The live viewer loop (webcam / video / image). Guarded by <code>__main__</code> .

This module **does not save landmark data** — it only detects and draws.

The extraction layer was built on top of it for that reason.

extract_dataset.py — extraction pipeline (~621 lines)

Symbol	Role
<code>build_models(opts)</code>	Create the enabled MediaPipe models for one clip.
<code>_raw_hands(hand_res)</code>	Hands without smoothing (used when <code>--no-smooth</code>).
<code>extract_clip(video_path, opts)</code>	Core: run detection over a clip -> stacked arrays + masks + meta.
<code>build_all_vector(arrays, opts)</code>	Concatenate enabled modalities into <code>(T, F)</code> ; return the layout.
<code>_proto(rows, with_vis)</code>	Rebuild a <code>NormalizedLandmarkList</code> from an array (for rendering).
<code>render_skeleton(arrays, masks, size, fps, out, opts)</code>	Black-bg skeleton video from saved arrays.
<code>qa_clip(arrays, masks, T, opts)</code>	Detection rates, ranges, NaN/Inf, <code>PASS/WARN/FAIL</code> verdict.
<code>discover(dir, only)</code>	Find <code><class>/<video></code> clips.
<code>main()</code>	Orchestrate: folders, loop, save, aggregate reports + README.

CLI flags: `--dataset --out --classes --limit --no-face --no-hands --no-pose --no-skeleton --no-smooth --proc-width --skip-existing --skeleton-size`.

verify_landmarks.py — verification + overlay (~313 lines)

Symbol	Role
<code>_proto(rows, with_vis)</code>	Rebuild a proto from an array.
<code>_present(block)</code>	Per-frame boolean: a block is "detected" if not all-zero.
<code>load_clip(out, cls, stem)</code>	Load whichever modality arrays exist.
<code>rebuild_all(arr, layout)</code>	Reconstruct the combined vector from the separate files.
<code>verify_clip(...)</code>	The 3 checks (structure, consistency, detection) + verdict.
<code>make_overlay(out, dataset, cls, stem)</code>	Side-by-side original+landmarks / skeleton video.
<code>main()</code>	Single-clip mode (<code>--video</code> , optional <code>--overlay</code>) or whole-dataset scan.

Consistency check is the strongest structural guarantee: it rebuilds `landmarks_all` from `landmarks_{pose, face, hands}` and asserts an exact match (`max diff` reported; observed `0.0`).

prune_clips.py — reversible curation (~115 lines)

Symbol	Role
<code>prune(pairs, out, dataset, excluded)</code>	Move source mp4 -> <code>excluded_clips/</code> , delete processed artifacts.
<code>refresh_reports(out, dropped)</code>	Drop rows from <code>manifest.csv</code> ; recompute <code>extraction_report.json</code> .
<code>main()</code>	CLI: <code>python prune_clips.py <class>/<stem> ...</code> (default: the 2 close-up clips).

Nothing is deleted irreversibly — sources are preserved in `excluded_clips/`.

dataset_report.py — infographics (~295 lines)

Symbol	Role
<code>load(out)</code>	Read all per-clip <code>metadata/*.json</code> .
<code>fig_overview</code>	Clips/class, verdict donut, mean detection, key numbers.
<code>fig_heatmap</code>	Class x modality detection-rate heatmap.
<code>fig_per_class</code>	Grouped detection bars per class.
<code>fig_frames</code>	Frames-per-clip histogram.
<code>fig_layout</code>	Composition of the <code>(T, 1692)</code> vector.
<code>fig_scorecard</code>	One-page verification scorecard (reads <code>verification_report.json</code>).
<code>main()</code>	Render all six PNGs into <code>processed_dataset/reports/</code> .

docs/make_diagrams.py & docs/build_pdf.py

Documentation tooling: `make_diagrams.py` renders the architecture / data-flow / verification diagrams; `build_pdf.py` parses these markdown files (plus the diagrams and infographics) into

TECHNICAL_DOCUMENTATION.pdf.

05 — Data Formats

Output folders

Folder	File	Shape	Contents
landmarks_pose/	<class>/<stem>.npy	(T, 33, 4)	x, y, z, visibility
landmarks_face/	<class>/<stem>.npy	(T, 478, 3)	x, y, z
landmarks_hands/	<class>/<stem>.npy	(T, 2, 21, 3)	slot 0 = Left, slot 1 = Right; x, y, z
landmarks_all/	<class>/<stem>.npy	(T, 1692)	combined per-frame feature vector
skeleton_videos/	<class>/<stem>.mp4	—	black-bg skeleton from the arrays
overlay_check/	<class>/<stem>.mp4	—	original + landmarks ■ skeleton
metadata/	<class>/<stem>.json	—	per-clip QA

T is the number of frames in the clip. dtype is float32 everywhere.

The combined vector (T, 1692)

Per frame, in this exact order (FEATURE_LAYOUT.json is the machine-readable source of truth):

Block	Columns	Width	Source
pose	0 – 131	132	33 x (x, y, z, visibility)
face	132 – 1565	1434	478 x (x, y, z)
left_hand	1566 – 1628	63	21 x (x, y, z)
right_hand	1629 – 1691	63	21 x (x, y, z)

Total **1692** features/frame. If a modality is disabled with `--no-*`, its block is omitted everywhere and the layout shrinks accordingly.

```
import numpy as np, json
all_vec = np.load("processed_dataset/landmarks_all/hello/hello.npy") # (T, 1692)
layout = json.load(open("processed_dataset/FEATURE_LAYOUT.json"))
pose_block = all_vec[:, 0:132].reshape(-1, 33, 4) # == landmarks_pose/hello/hello.npy
```

Coordinate system

- x, y — normalised image coordinates, [0, 1] = fraction of width / height.
- z — relative depth, same scale as x (negative = closer to camera).
- visibility (pose only) — [0, 1] confidence the joint is visible.
- Missing landmarks in a frame are stored as zeros; presence is recorded

in the metadata masks (and is recoverable as "block is not all-zero").

metadata/<class>/<stem>.json

```
{
  "class": "hello", "label": 1, "stem": "hello",
  "source": "dataset/hello/hello.mp4", "source_fps": 30.0,
  "frames_read": 65, "frames_skipped": 0, "T": 65,
```

```

    "smoothed": true, "proc_width": 960,
    "shapes": { "pose": [65,33,4], "face": [65,478,3], "hands": [65,2,21,3] },
    "all_shape": [65, 1692],
    "masks": { "pose": [1,1,...], "face": [...], "hands": [[0,1],...] },
    "qa": {
      "detection_rate": { "pose": .94, "face": .94, "left_hand": 0, "right_hand": .46, "any_hand": .46 },
      "ranges": { ... }, "issues": [], "verdict": "PASS"
    }
  }
}

```

Dataset-wide files

- `classes.json` — { "good": 0, "hello": 1, ... } (class -> integer label).
- `manifest.csv` — one row per clip: `class, label, stem, T, verdict,

`all_npy, skeleton`. Use this to drive a train/val/test split.`

- `extraction_report.json` — `n_clips`, `verdicts`, `mean_detection_rate`,

per-clip summaries (produced by extraction).

- `verification_report.json` — independent re-check: per-clip

`all_vs_parts_max_diff`, detection rates, verdict (produced by verification).

Verdicts

Verdict	Meaning
PASS	No issues: valid floats, consistent, pose detected in $\geq 50\%$ of frames.
WARN	Usable but flagged (e.g. pose $< 50\%$, or face/hands far outside <code>[0,1]</code>).
FAIL	Unusable: empty clip, or NaN/Inf, or <code>all</code> $\neq$ parts.

06 — Developer Guide

Setup

```
pip install mediapipe opencv-python numpy matplotlib reportlab
```

Python 3.11 recommended (MediaPipe 0.10.9 wheels).

Everyday commands

```
# Extract the whole dataset (downscale to 960px wide for speed)
python extract_dataset.py --out processed_dataset --proc-width 960

# Extract just some classes / a few clips (quick test)
python extract_dataset.py --classes hello you --limit 2

# Verify every .npy file (structure + consistency + detection rates)
python verify_landmarks.py --out processed_dataset

# Eyes-on proof for one clip (landmarks reprojected on the original video)
python verify_landmarks.py --out processed_dataset --video hello/hello --dataset dataset --overlay

# Generate overlays for ALL clips
python verify_landmarks.py --out processed_dataset --dataset dataset --overlay

# Drop clips (reversible) and refresh reports
python prune_clips.py no/no_13 yes/yes_12

# Rebuild the infographics
python dataset_report.py

# Live viewer (sanity check the detector itself)
python landmark_detector.py --source dataset/hello/hello.mp4
```

Common tasks

Re-extract only one modality

```
python extract_dataset.py --no-face --no-pose # hands only
```

The combined `landmarks_all` and `FEATURE_LAYOUT.json` adjust automatically.

Add a new class

1. Drop `dataset/<new_class>/*.mp4`.
2. `python extract_dataset.py --classes <new_class>` (or re-run all).
3. `python verify_landmarks.py --out processed_dataset`.
4. `python dataset_report.py`.

Resume an interrupted run

```
python extract_dataset.py --skip-existing
```

Store raw (unsmoothed) landmarks

```
python extract_dataset.py --no-smooth
```

Debugging guide

Symptom	Likely cause / action
A clip is WARN for "pose < 50%"	Body/face out of frame (close-up). Inspect with <code>--overlay</code> ; prune if it's an outlier.

Symptom	Likely cause / action
<code>left_hand</code> near 0%	Expected for one-handed (right-handed) signs — see the heatmap. Not a bug.
Pose <code>x/y</code> > 1	By design — MediaPipe extrapolates off-screen joints. Pose is not range-checked.
FAIL for "all != parts"	Corruption or a format change. Re-extract that clip.
Slow extraction	Lower <code>--proc-width</code> (e.g. 640); it does not affect normalised coordinates.
Hands swapped L/R	Check <code>anatomical_label</code> / <code>match_hands_to_sides</code> ; raw video is <i>*not*</i> mirrored.

Inspect a `.npy` by hand

```
import numpy as np
a = np.load("processed_dataset/landmarks_pose/hello/hello.npy")
print(a.shape, a.dtype, np.isnan(a).any())
print("pose detected frames:", int((np.abs(a).reshape(len(a), -1).sum(1) > 0).sum()), "/", len(a))
```

Performance notes

- Models are **recreated per clip** (≈ 0.5 – 1 s each) to avoid cross-clip tracking leakage; this dominates the per-clip cost on short videos.
- `--proc-width` is the main speed knob. Normalised landmark coordinates are resolution-independent, so downscaling input is essentially free accuracy-wise for upper-body framing.
- Skeleton/overlay rendering reads only the saved arrays (no detection), so it is fast; disable with `--no-skeleton` if not needed.

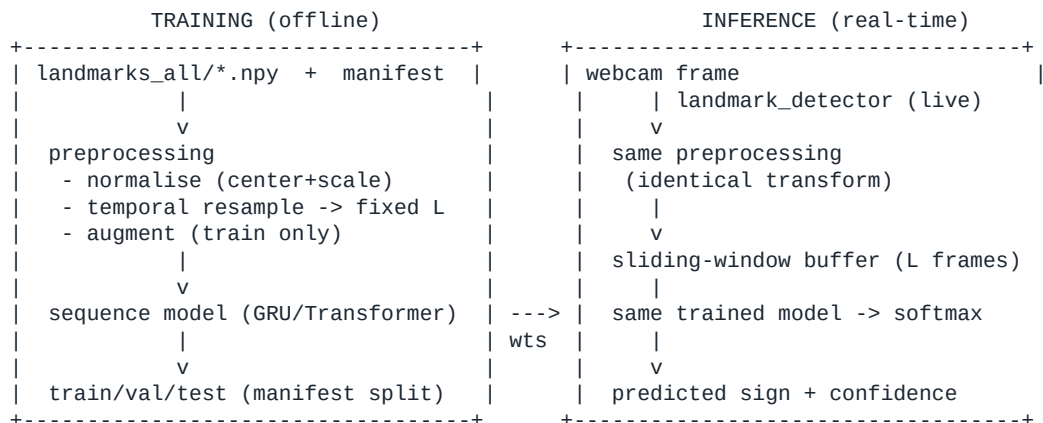
Conventions for contributors

- Keep detection/drawing logic in `landmark_detector.py`; tools should import it, not re-implement it.
- Never hand-edit files under `processed_dataset/` — regenerate them.
- Prune by **moving** sources to `excluded_clips/`, never deleting `dataset/`.

07 — Future System Design

The current system ends at a verified landmark dataset. This is the suggested design for the next phases: **training** and **real-time inference**. None of it requires changing the extraction format — it consumes `landmarks_all/*.npy` and `manifest.csv` read-only.

Proposed end-to-end system



The key property: **training and inference share one preprocessing function and one model**, so what you train on is exactly what you serve.

1. Preprocessing (new module, e.g. `features.py`)

A single function `to_features(all_vec) -> (L, D)` used by both training and inference:

- **Spatial normalisation** (translation/scale invariance): per frame, subtract

a reference point (mid-shoulder from pose) and divide by a reference length (shoulder width). Makes signs invariant to where/how the signer is.

- **Temporal resampling**: clips are 14–286 frames; resample each to a fixed

length `L` (e.g. 48) by uniform sampling or interpolation. Alternatively pad + mask for a variable-length model.

- **Mask channel**: append the presence mask so the model can tell "hand absent"

from "hand at origin".

- Optionally **drop the face block** (1434 of 1692 dims) for a lighter model if

facial expression isn't needed for the target signs.

2. Model

- Baseline: **2-layer BiGRU** or **small Transformer encoder** over `(L, D)` ->

mean/attention pool -> linear -> 12-way softmax. Skeleton-based recognition needs far less data/compute than pixel models.

- Loss: cross-entropy; report per-class accuracy + confusion matrix (the heatmap

already shows which signs share hand structure and may confuse).

3. Data splitting & balancing

- Drive the split from `manifest.csv` (stratify by `class`). With ~10–14 clips

per class, use **k-fold cross-validation** rather than a single split.

- Augmentation (train only): small rotations/scales/jitter on coordinates, time warping, horizontal flip **with L/R hand-slot swap** (turns a right-handed example into a left-handed one).

4. Real-time inference

Reuse `landmark_detector.run()`'s per-frame detection, feed each frame through the **same** `to_features`, keep a sliding window of `L` frames, and classify continuously. Smoothing and hand-side matching are already solved in the core library.

5. Scaling & robustness (when the dataset grows)

- **Parallel extraction:** `extract_clip` is independent per clip — shard by class across processes; `--skip-existing` makes runs resumable.
- **Quality gates:** promote QA to a hard gate (e.g. auto-exclude pose < 50%) via `prune_clips.py` driven by `extraction_report.json`.
- **More signers / lighting:** collect varied data; the normalisation step keeps features comparable across sources.
- **Two-handed coverage:** current data is mostly right-handed; flip-augmentation and balanced collection will improve left-hand generalisation.

Suggested next deliverables

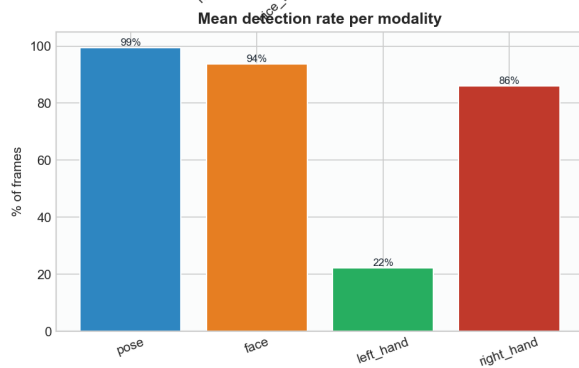
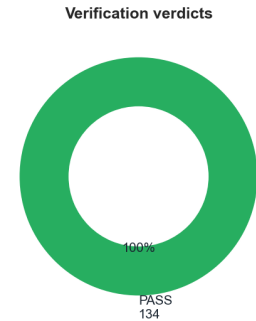
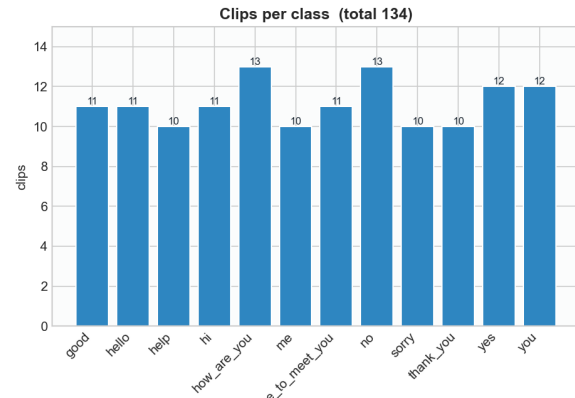
1. `features.py` — shared preprocessing (normalise + resample + mask).
2. `train.py` — k-fold training over `landmarks_all`, writes a model + metrics.
3. `infer_live.py` — real-time classifier built on `landmark_detector`.
4. Extend `dataset_report.py` with a confusion-matrix panel once a model exists.

Appendix — Verification Infographics

Auto-generated charts from `dataset_report.py` summarising extraction and verification quality.

01 Dataset Overview

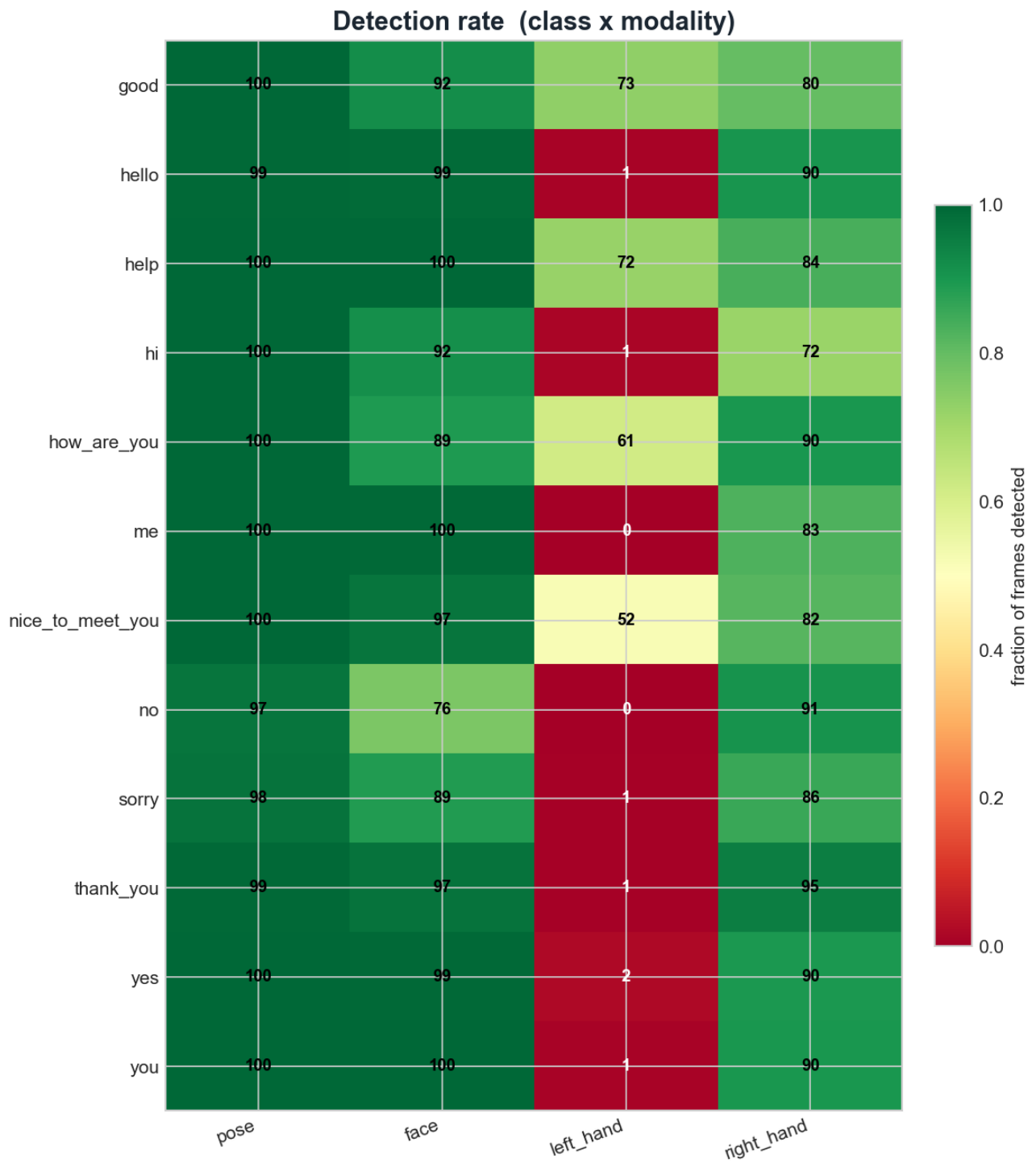
Sign-Language Landmark Dataset — Overview



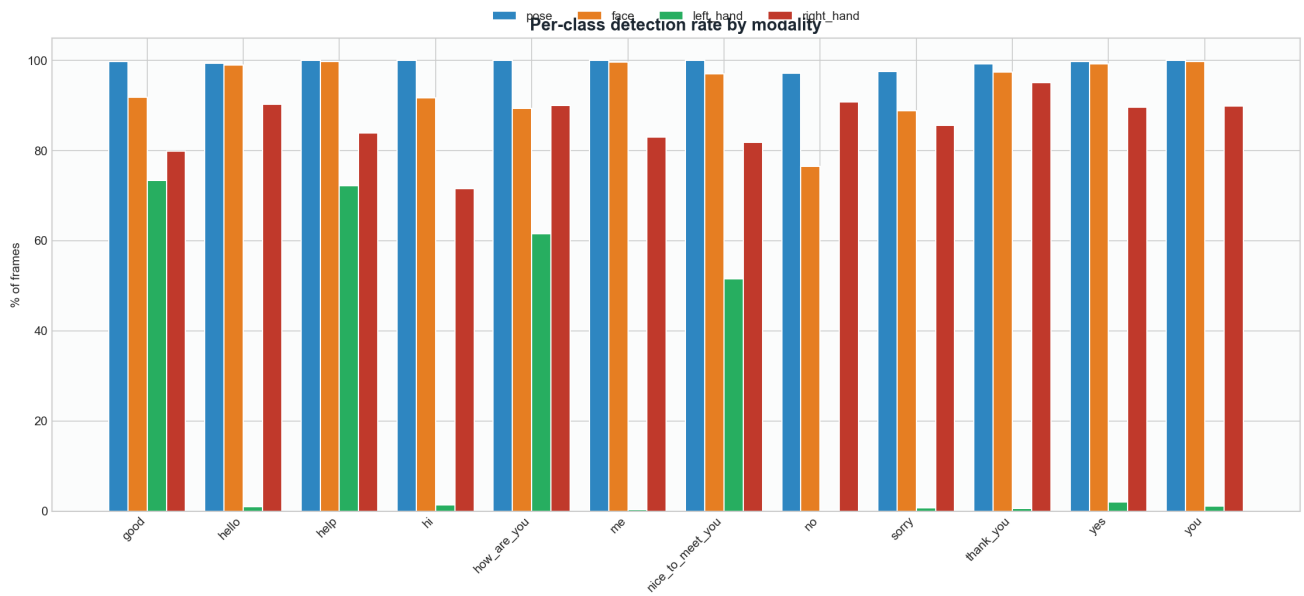
Key numbers

Classes	12
Clips	134
Total frames	9,417
Frames / clip	14–286 (median 60)
Feature vector	1692 dims / frame
All == parts	max diff 0.0 (exact)
NaN / Inf	none

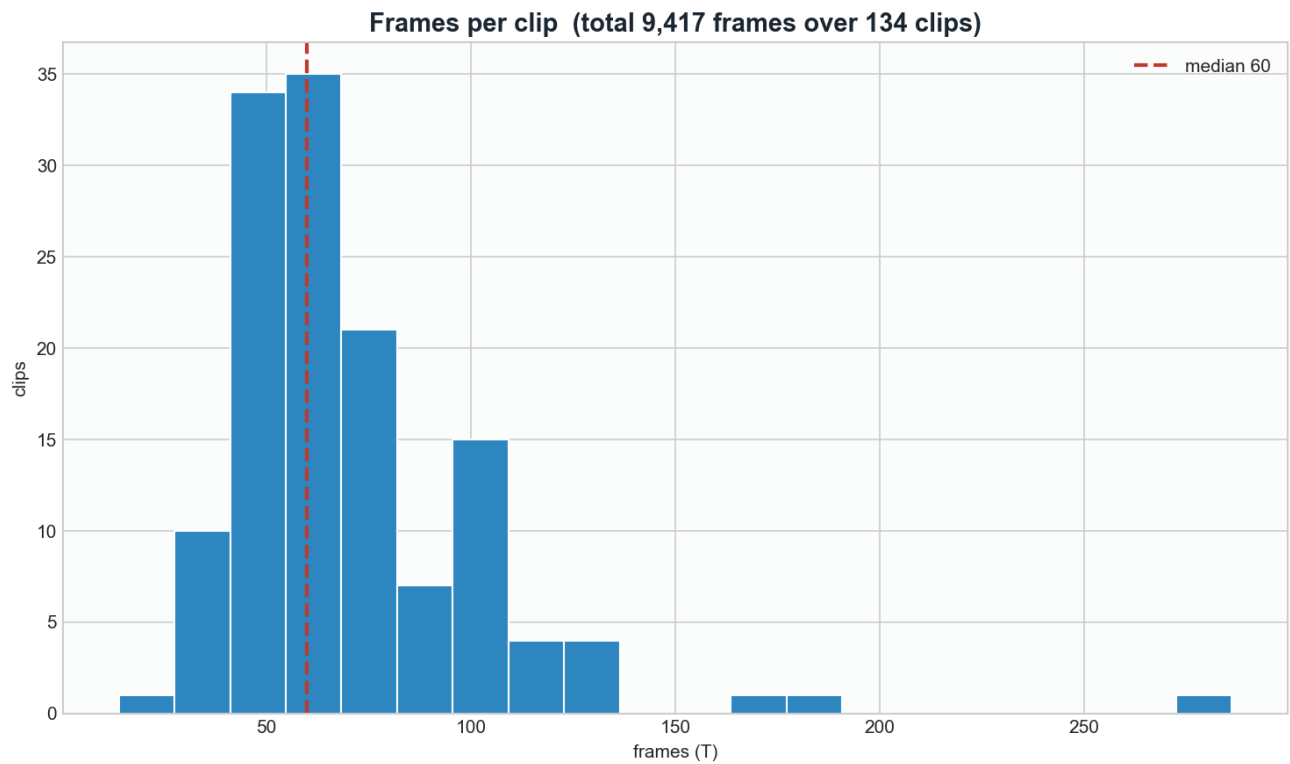
02 Detection Heatmap



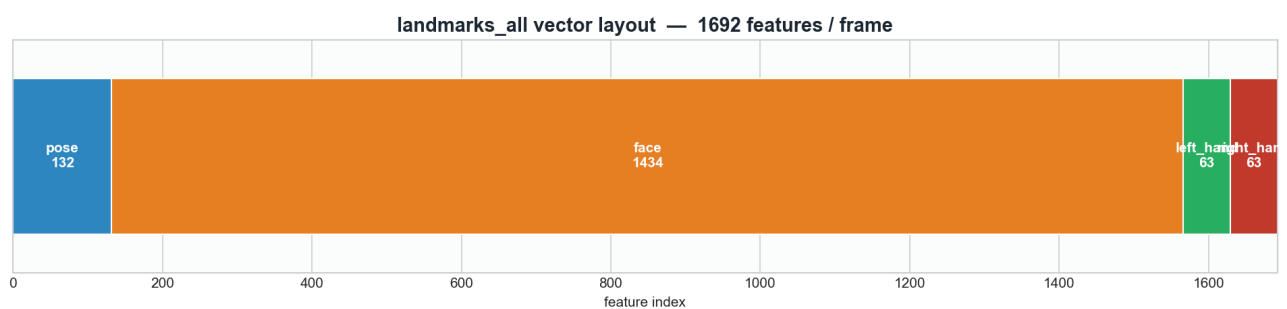
03 Per Class Detection



04 Frame Distribution



05 Feature Layout



06 Verification Summary

VERIFICATION SCORECARD

Clips verified	134	PASS
Structure (shape / dtype)	all valid float32	PASS
NaN / Inf present	none	PASS
Consistency all == face+hands+pose	max diff 0.0e+00	PASS
Verdicts	PASS 134 / WARN 0 / FAIL 0	PASS
Pose detection (mean)	99%	PASS
Face detection (mean)	94%	PASS
Any-hand detection (mean)	87%	PASS

Excluded as framing outliers (hand close-ups, face/body off-frame): no/no_13, yes/yes_12. Sources preserved in excluded_clips/.